

PROJECT EXPLAINER

Extending FRANQ with Dependency-Aware Entity–Attribute Verification and Targeted Self-Correction for Faithful Retrieval-Augmented Generation

A complete, plain-language guide to the project — from the problem it solves to how it works, step by step.

Prepared for: First-time readers of this research project

Contents

- 1. Introduction – What is this project?
- 2. Problem Statement – What is broken, and why does it matter?
- 3. Research Gap – What is missing in existing work?
- 4. Proposed Methodology – The improved, strong novelty
- 5. Algorithms Used – What tools we use, and why
- 6. Modules of the System – How the code is organised
- 7. How It All Works Together – A walk-through example
- 8. Conclusion
- 9. A-Z Quick Recap

1. Introduction — What is this project?

Large Language Models (LLMs), like the AI chatbots many of us use today, are very good at writing fluent, confident-sounding answers. But sometimes they say things that are simply not true. This is called **hallucination**. To fix this, researchers built a technique called **Retrieval-Augmented Generation (RAG)** — instead of the AI making up an answer purely from memory, it first looks up real documents (like searching a library) and then writes its answer based on what it found.

Even with RAG, the AI can still get things wrong — it might misread the documents, mix up facts, or say something the documents never actually supported. A recent framework called **FRANQ** (a real published research paper) tackles this by checking two separate things: is the answer faithful to the documents it retrieved, and is the answer factually true in the real world? FRANQ picks a different checking method depending on which of these two problems is more likely.

Simple analogy: Think of an AI answering questions like a student writing an essay using library books. 'Faithfulness' means: did the student actually use what the books said, or did they make it up? 'Factuality' means: are the books themselves (and the student's reading of them) actually correct? FRANQ is like a strict teacher who checks both things — but always in the same fixed way, no matter which student or which essay.

This project takes FRANQ and makes it smarter and more complete in three ways: it checks facts more carefully by understanding how facts about the same topic are connected to each other, it learns (instead of following a fixed rule) how to decide which checking method to use, and — most importantly — it doesn't just point out mistakes, it actually goes and **fixes** them, then double-checks that the fix didn't break anything that was already correct.

2. Problem Statement — What is broken, and why does it matter?

FRANQ is a strong idea, but it has three honest weaknesses — and the FRANQ authors themselves point some of these out.

Problem 1: Facts are checked one at a time, as if they don't know each other

FRANQ (and similar systems) break a sentence into small, separate facts and check each one on its own. But real facts about a person or thing are often connected. If the system gets the main subject wrong, every fact attached to it becomes shaky too — yet a flat, one-fact-at-a-time checker cannot see that connection.

Simple analogy: Imagine a family tree. If you get the name of the grandfather wrong, then everything you say about his children and grandchildren also becomes questionable. Checking 'this fact' and 'that fact' completely separately misses this — it's like checking each leaf of a tree without ever looking at the branch it's attached to.

Problem 2: The decision rule for which check to run is fixed and rigid

FRANQ uses a simple IF-ELSE rule: if the answer looks unfaithful, use one checking method; otherwise, use another. This fixed rule cannot adapt — it treats every situation the same way, even though real cases vary a lot in how confident the system should be.

Simple analogy: It's like a teacher who always gives the exact same test to every student, regardless of whether the student is confused about spelling or confused about maths. A smarter teacher would look at the situation and choose the right kind of test each time.

Problem 3: Systems detect mistakes, but usually don't fix them

Most existing systems, including FRANQ, stop at saying 'this looks wrong.' They rarely go the next step and correct the mistake. And the few systems that do attempt correction don't check whether their fix accidentally damaged a fact that was already correct.

Simple analogy: This is like a doctor who tells you 'something in your body is wrong' but never treats it — and if they do treat it, they never check if the medicine caused a new problem elsewhere.

Why this matters: RAG systems are increasingly used in serious settings — medicine, law, customer support, education. A wrong fact that goes undetected, or a 'fix' that quietly introduces a new error, can cause real harm. Solving all three problems together makes RAG systems noticeably more trustworthy.

3. Research Gap — What is missing in existing work?

Before deciding what to build, existing research was studied carefully to make sure this project adds something genuinely new, and does not repeat work that already exists.

Existing Work	What it does	What it is missing
FRANQ (base paper)	Separates faithfulness vs. factuality checking using a fixed rule.	Flat fact checking, fixed decision rule, no correction step.
RAC (2025)	Breaks answers into small independent facts and corrects wrong ones.	Treats facts as independent (no entity–attribute links); no FRANQ-style calibrated routing; no safety check after correction.
QuCo-RAG / HalluEntity / RHD (2026, concurrent)	Focus on detecting unsupported or incorrect entities/claims.	Detection only — they flag problems but do not correct them.

The gap, in one sentence: no existing system combines entity–attribute dependency-aware checking, a learned calibrated decision mechanism, and a safety-checked correction loop, all within FRANQ's faithfulness-and-factuality framework. That combination — and especially the safety-checked correction — is the open gap this project fills.

4. Proposed Methodology — The Improved, Strong Novelty

The project's contribution rests on three connected pillars. Each one directly answers one of the three problems described earlier, and together they form a single coherent pipeline — not three unrelated add-ons.

Pillar 1: Dependency-Aware Entity–Attribute Verification Graph

Instead of breaking a sentence into flat, disconnected facts, this project builds a small graph for each entity (a person, place, or thing) that connects it to all of its attributes (its properties, like a birth year or a birth city). Facts that belong to the same entity are checked together, so an error in the entity itself, or a suspicious pattern across its attributes, can be caught — something flat fact-checking cannot do.

Simple analogy: Example: the sentence 'Einstein was born in 1879 in Munich' is turned into a small tree: Einstein → birth year → 1879, and Einstein → birth city → Munich. If the system starts doubting the entity 'Einstein' himself, it now knows to look more carefully at both attached facts — not just one fact in isolation.

Pillar 2: Learned, Calibrated Uncertainty Router

FRANQ's original rule is a fixed IF-ELSE switch. This project replaces that switch with a small trained model (a 'router') that looks at several signals together — the faithfulness score, retrieval confidence, the type of entity, how strongly its attributes depend on each other, and uncertainty estimates — and then makes a calibrated decision about which verification strategy to use. 'Calibrated' means the confidence numbers it outputs can be trusted to actually reflect how likely it is to be right.

Simple analogy: Instead of a teacher always giving the same test, this is a teacher who looks at each student individually and picks the right kind of question for their specific weak spot — and who is honest about how sure they are before deciding.

Pillar 3: Budget-Bounded Correction Loop with Correction-Regret Safety Metric

When a fact is flagged as likely wrong, the system does not just report it — it runs a focused, targeted search (for example, searching only 'Einstein birth city' instead of redoing the whole answer), retrieves better evidence, and replaces only the incorrect part. This happens within a limited 'budget' of retrieval attempts, so the system doesn't search forever. After correcting, the system re-checks the whole answer using a new safety metric called **correction regret**: it measures whether the fix accidentally damaged a fact that was already correct.

Simple analogy: Correction Regret = (number of facts that were correct before, but got broken during fixing) ÷ (total number of facts that were correct before). A good doctor doesn't just treat the one symptom you came in with — they check the medicine didn't cause a new problem elsewhere. This metric is that check, applied to fact-correction.

Why this is a strong novelty: RAC already corrects facts, and detection-focused papers (QuCo-RAG, HalluEntity, RHD) already flag errors well — but none of them combine dependency-aware checking, a learned router, and a safety-checked correction loop together. The correction-regret metric in particular does not exist in any of the compared systems, making it the most distinctive, checkable point of difference in this project.

Official Contribution Statement (journal-ready wording)

This work extends FRANQ through three integrated contributions. First, it introduces a dependency-aware entity–attribute verification graph that explicitly captures dependencies among facts associated with the

same entity, rather than treating factual units as independent as in RAC. Second, it develops a learned and calibrated uncertainty router that replaces FRANQ's fixed switching rule between faithfulness- and factuality-oriented signals with adaptive, confidence-aware verification. Third, it proposes a budget-bounded targeted self-correction loop guided by a novel correction-regret metric, which quantifies whether the correction process inadvertently damages facts that were initially correct. Together, these components extend hallucination handling from detection toward dependency-aware, uncertainty-calibrated, and safety-conscious correction. In contrast to the evaluated baselines — including RAC, QuCo-RAG, HalluEntity, and RHD — the proposed framework explicitly evaluates the risk of collateral damage caused by factual correction.

5. Algorithms Used — What tools we use, and why

Each part of the pipeline uses a specific, well-established technique, chosen because it is the right tool for that particular job. None of these are invented from scratch — the novelty is in how they are combined.

Algorithm / Component	What it does	Why it is used here
LLM-based Fact Extractor	Reads the generated answer and pulls out individual entity–attribute–value facts.	Needed to turn free-flowing text into structured facts the rest of the pipeline can check one by one.
Dependency-Aware Entity–Attribute Graph Builder	Links facts that share the same entity into a small graph instead of a flat list.	Lets the system see when one entity error puts several attached facts at risk — this is Pillar 1.
Dense Retriever	Searches a document collection using meaning-based (not just keyword) matching to find relevant evidence.	Provides the evidence used both for the original answer and for focused re-checks during correction.
AlignScore	A trained metric that scores how well a piece of text is actually supported by given evidence.	Used to measure faithfulness — whether the answer matches what the retrieved documents actually say.
Semantic Entropy (Uncertainty Signal)	Measures how much the model's own answers vary in meaning when asked the same thing multiple ways.	High variation signals the model itself is unsure — a key input to the calibrated router (Pillar 2).
Parametric Knowledge UQ	Estimates how confident the model is based on what it learned during training, separate from retrieval.	Distinguishes 'the model itself doesn't know' from 'the retrieved documents were unclear' — both matter differently.
Learned Calibrated Router	A trained model that combines all the above signals into one calibrated decision.	Replaces FRANQ's fixed IF-ELSE rule with a decision that adapts to each specific case — Pillar 2.
Targeted Correction & Re-verification Loop	Runs a focused search for only the suspicious fact, retrieves new evidence, replaces the value, and re-checks it.	Fixes only what's broken, within a limited search budget, instead of regenerating the whole answer — Pillar 3.
Correction-Regret Metric	Compares the set of correct facts before and after correction to spot any new damage.	The safety check that makes correction trustworthy, not just aggressive — Pillar 3.

6. Modules of the System — How the code is organised

The implementation is built as a Python package called **franq_ext**. It is organised so that every module maps directly onto one part of the paper — this makes it easy to check that what is written in the paper is exactly what the code does.

Module	Role in the pipeline
Extraction module	Uses the LLM-based extractor to convert the generated answer into entity–attribute–value facts.
Graph construction module	Builds the dependency-aware entity–attribute graph linking related facts (Pillar 1).
Retrieval module	Wraps the dense retriever to fetch supporting evidence, both initially and during targeted correction.
Faithfulness scoring module	Runs AlignScore to measure how well each fact is supported by retrieved evidence.
Uncertainty quantification module	Computes semantic entropy and parametric knowledge uncertainty signals.
Calibrated router module	The learned model that takes all signals and decides the verification strategy (Pillar 2).
Correction module	Performs the budget-bounded targeted correction loop: focused search, evidence check, value replacement (Pillar 3).
Re-verification module	Re-checks the corrected answer and computes the correction-regret metric.
Evaluation / experiments module	Six sequential scripts covering baseline reproduction through to final results computation.

The package also includes unit tests (small checks that confirm each module works correctly on its own) and a runnable end-to-end demo (a working example that shows the whole pipeline running from start to finish on a sample input).

7. How It All Works Together — A Walk-Through Example

Here is the full pipeline in order, using the Einstein example throughout:

- **Step 1 — Answer generation:** The AI generates the sentence 'Einstein was born in 1879 in Munich' using retrieved documents.
- **Step 2 — Extraction:** The extraction module pulls out two facts: (Einstein, birth year, 1879) and (Einstein, birth city, Munich).
- **Step 3 — Graph building:** The graph module links both facts under the entity 'Einstein', so they are checked as a connected pair, not in isolation.
- **Step 4 — Scoring:** AlignScore checks how well each fact matches the retrieved evidence; semantic entropy and parametric UQ estimate how confident the model itself is.
- **Step 5 — Routing decision:** The calibrated router looks at all these signals and decides: 'birth year' looks fine, but 'birth city' looks suspicious and needs deeper checking.
- **Step 6 — Targeted correction:** The correction module runs a focused search for 'Einstein birth city', finds that the correct answer is Ulm (not Munich), and replaces only that one value.
- **Step 7 — Re-verification and safety check:** The system re-checks the whole answer. It confirms 'birth year = 1879' is still correct (unchanged) and 'birth city' is now fixed to Ulm. Correction regret = 0, because no previously-correct fact was damaged.

This example shows all three pillars working as a single pipeline: the graph (Pillar 1) made sure both facts about Einstein were considered together, the router (Pillar 2) made a smart, calibrated decision about which fact needed checking, and the correction loop with its safety metric (Pillar 3) fixed the problem and proved nothing else was broken in the process.

8. Conclusion

This project extends FRANQ, a real published RAG-faithfulness framework, by directly addressing three weaknesses that the framework itself is known to have: flat, disconnected fact-checking; a rigid, fixed decision rule; and the absence of any actual correction mechanism. As stated formally in the official contribution statement (Section 4), the framework moves hallucination handling from mere detection toward dependency-aware, uncertainty-calibrated, and safety-conscious correction — and is the only evaluated approach that explicitly measures the risk of collateral damage caused by its own correction process.

A careful check of the closest related work — RAC (which corrects facts, but treats them as independent and has no safety metric) and the concurrent 2026 detection-focused systems QuCo-RAG, HalluEntity, and RHD (which detect problems but never fix them) — confirms that no existing system combines all three pillars. This makes the correction-regret metric, in particular, a genuinely distinctive and easily verifiable contribution.

The idea itself is assessed as strong and well-motivated (rated 8.5/10 on novelty), and is scoped appropriately for a standard peer-reviewed journal (such as IEEE Access), rather than requiring an unrealistically large number of new contributions. The remaining and most important step is execution: running the experiments, generating results, and confirming that the entity–attribute graph, the learned router, and the correction loop each measurably outperform their FRANQ-style baselines. Once this evidence is in place, the project is well-positioned for a strong first submission.

9. A-Z Quick Recap

Term	In one line
AlignScore	Measures if a fact is supported by retrieved evidence.
Budget-bounded correction	Correction is allowed only a limited number of search attempts, to stay efficient.
Calibrated router	A learned model that picks the right checking method, with trustworthy confidence.
Correction regret	Measures if fixing one fact accidentally broke a fact that was already correct.
Dependency-aware graph	Links facts about the same entity instead of checking them in isolation.
Entity–attribute	An entity (e.g. Einstein) and one of its properties (e.g. birth city).
Faithfulness	Whether the answer matches what the retrieved documents actually say.
Factuality	Whether the answer is actually true in the real world.
FRANQ	The base research paper this project extends.
franq_ext	The name of this project's Python code package.
Hallucination	When an AI states something false or unsupported.
Parametric knowledge UQ	Confidence estimate based on what the model learned in training.
RAC	A real, related 2025 paper that corrects independent atomic facts.
RAG	Retrieval-Augmented Generation — the AI looks up documents before answering.
Semantic entropy	Measures how much the model's own answers disagree with themselves.
Targeted correction	Fixing only the one wrong fact, instead of regenerating the whole answer.